

Rajesh Chandolu

Senior Software Developer

github.com/rajeshkumarchandolu

rajeshkumarchandolu@gmail.com

+91 8185840307

Guttala Begumpet,

Hyderabad

Telangana- 500081

Summary

Platform Engineer with **11+ years** of experience in Cloud Storage Infrastructure and Distributed Systems at Scale. Demonstrated experience in designing and delivering large-scale block storage solutions, including fault domain architecture, performance optimization, and availability improvements **serving millions of OCI customers**. Experienced in building and scaling cloud-native microservices across control plane, management plane, and data plane. Skilled in leading technical design discussions, preparing engineering change reviews, and driving cross-team initiatives from concept to production, with hands-on experience in **handling and mitigating Sev1 incidents for Tier-0 block storage services**.

Education

Bachelor of Technology

Vasireddy Venkatadri Institute of Technology, Nambur, Andhra Pradesh

05/2015

Completed B.Tech in Information Technology with First Class

Skills

Programming Languages: JAVA, Python, C++, SCALA, JAVASCRIPT.

Distributed Systems: Fault Tolerance, Distributed Workflows, Rate Limiting, Circuit Breaker

Cloud Infrastructures: AWS, OCI

DevOps: Docker, Kubernetes, Helm, Terraform

Backend Frameworks: Spring Boot, Dropwizard, Netty-GRPC

Storage Protocols: iSCSI, NVMe

Build Tools: Maven, Gradle, Bazel

Monitoring: Grafana, Splunk, Lumberjack

Experience

Senior Software Engineer

Oracle India Pvt Ltd • Hyderabad, Telangana

07/2021 - Present

Product: Oracle Cloud Infrastructure (OCI)

Projects undertaken:

- **Volume Creation Performance at Scale**

The perceived end-to-end latency for volume creation was identified as a critical gap for tenants where launch time is business critical, despite the system efficiently **processing millions of requests** through asynchronous pipelines. Contributed to the 100RPS initiative by implementing tenant prioritization and analyzing tenant request patterns to **predictively pre-create volume instances before tenant requests arrive**, ensuring immediate response times. Additionally delivered key enhancements in the block storage control path including adaptive cloning, which eliminated the dependency on object storage during volume creation from backup by creating an internal replica volume and provisioning directly from it, saving bandwidth and reducing latency, and fast cloning, which introduced a duplicate replica mechanism for smaller tenants frequently cloning the same image where only newly written blocks are stored while remaining blocks are read from the parent replica, reducing storage overhead and improving clone creation speed. These enhancements collectively resulted in immediate response times for volume creation, significantly improving the end-to-end customer experience and meeting the 100RPS performance target.

- **Block Storage Workflow Rate Limiting & Prioritization**

Identified a scalability gap where a single tenant could act as a **noisy neighbour** by overwhelming shared asynchronous workflows, impacting other tenants. Implemented a per-tenant rate limiter to prevent any single tenant from choking the workflow processing pipeline, ensuring fair resource

distribution across all tenants and **significantly reducing the blast radius during incidents.**

Additionally introduced a **weighted priority system** for workflows, assigning higher priority to critical operations such as volume creation and backup creation, while allowing lower priority operations like volume deletion and backup deletion to wait, ensuring optimal resource utilization and improved customer experience.

- **Block Storage Control Plane Circuit Breaker**

Implemented a circuit breaker pattern in the block storage control plane using the **Failsafe SDK** to handle failures in dependent services such as Networking and Management Plane. When a dependent service becomes unavailable, the circuit breaker enables **fast failure** for only the affected operations, freeing up workflow capacity for unaffected operations to continue processing. For example, if volume creation APIs are failing due to a downstream dependency, backup and other independent workflows continue to execute without disruption, significantly **reducing the blast radius** of downstream failures and improving overall system resilience

- **Compute Aligned Fault Domains**

Designed and led the compute aligned fault domains solution to address system vulnerabilities introduced by the OCI-on-OCI overlay. Prepared and presented the ECAR document to align stakeholders on the approach. Redesigning and enhancing multiple core components to align with compute fault domains including volume placement logic, attachment placement, and repair scenarios, ensuring fault domain awareness across the entire block storage stack. Further enhanced the substrate by splitting nodes into 3 fault domains, enabling zero-downtime deployments and **reducing the deployment window from 7 days to 1 day**, significantly improving deployment speed and operational efficiency.

- **Block Storage Backup Improvements**

Identified and resolved critical bottlenecks in the block storage backup pipeline that leverages Object Storage for volume block backups. Recognized that Object Storage enforces rate limits at the bucket level, which was causing contention due to shared usage across multiple tenants and internal services. Redesigning the backup logic to distribute load across tenant-specific buckets, taking advantage of existing sharding and load balancing provided by the Object Storage team, significantly improving throughput. Additionally optimized the upload chunk size from 4KB to 16KB, **reducing Object Storage API calls by ~50%** and reducing overall **backup time by 30%**.

- **Block Storage Management Plane - NVMe Attachment APIs**

Led the API specification design discussions for NVMe attachment lifecycle in the block storage management plane, with all design proposals being reviewed and accepted by the team. Implemented the CRUD APIs (create, update, delete) invoked by the Block Storage Control Plane (BSCP), along with multipath attachment support for NVMe connections. This significantly reduced customer-reported tickets related to manual and automatic iSCSI connection configuration issues during system restarts and incidents.

- **Block Storage AArch64 Architecture Support**

Led the enablement of ARM64 (AArch64) architecture support for block storage, which previously relied solely on x86_64 binaries. Identified and resolved a critical dependency on internal frameworks such as [crcutil](#) that were incompatible with AArch64, replacing them with cross-platform alternatives like Google [CRC32C](#) to ensure full compatibility. Implemented the necessary build and deployment changes to support running block storage on AArch64 instances, enabling the organization to leverage a strategic contract with Ampere for significantly reduced storage server costs.

- **Block Storage Dynamic Performance Scaling**

Collaborated with the team to design and implement the dynamic performance scaling feature with autotuning, working across all planes — control plane, management plane, and data plane — to deliver end-to-end implementation. The feature enables automatic adjustment of volume performance between a specified default and maximum VPU/GB threshold based on real-time I/O metrics monitored by Block Volume. Subsequently enhanced the feature by **introducing noisy neighbor guardrails**, limiting aggressive auto-scaling to protect shared infrastructure and ensure fair performance distribution across tenants.

- **iSCSI Login Path Enhancement**

Identified a gap in the iSCSI login path where invalid login attempts, such as expired KMS keys, were

handled by abruptly closing the socket connection. This caused compute machines to treat the failure as a network issue, resulting in redundant retry calls and unnecessary kernel error logs on the customer side. Analyzed the iSCSI specification and implemented the correct login response codes, ensuring compute machines correctly interpret authentication failures without triggering retries. This eliminated noise in customer kernel logs, improving customer experience and reducing confusion during incidents.

- **Block Storage CI/CD Pipeline Migration**

Migrated the build pipeline from TeamCity to OCI internal service pipelines, optimizing the build process by mounting Maven jars which reduced build times by 2x. Additionally automated the test cluster deployments for daily builds, enabling continuous validation of test failures with automated notifications to respective Slack channels, improving team productivity and reducing manual intervention in the build and release process.

- **Block Storage Availability Enhancements**

Developed a multi-region data aggregation tool to monitor and report overall service availability across all OCI regions. Analyzed critical bottlenecks in the system and resolved key bugs that significantly reduced 4xx/5xx error rates, improving overall service reliability and stability.

Platform Engineer

Paysafe Inc

04/2020 - 07/2021

Product: Customer Vault

Projects undertaken:

- **Customer Vault Microservice Migration**

Contributed to the migration of Customer Vault APIs from a monolithic WAR deployment to a standalone Spring Boot microservice architecture, improving scalability, maintainability and cloud readiness of the application

- **Shadow Testing**

Implemented shadow testing infrastructure for validating the Customer Vault microservice migration by leveraging and customizing the Twitter open-source [Diffy](#) project to meet internal requirements. Configured Diffy to capture and compare API responses between the legacy and new microservice, surfacing any diffs for validation. Additionally set up the complete shadow testing infrastructure in the staging cluster, enabling continuous end-to-end validation of all Customer Vault APIs before production deployment.

Software Engineer

Kony India Private Limited

04/2015 - 04/2020

Product: Middleware

Projects undertaken:

- **Kony Fabric Connectors**

Designed and developed multiple data adapters for third-party services including Braintree and Microsoft Graph, abstracting the underlying API complexity and enabling customers to integrate these services into their mobile applications simply by providing configuration parameters. This significantly reduced the integration effort for customers, providing an out-of-the-box connectivity experience within the Kony platform.

- **Kony Marketplace Components**

Developed multiple reusable UI components for the Kony Marketplace including Calendar, Login, and Hamburger Menu templates, enabling developers to accelerate mobile application development by leveraging pre-built, configurable components. The Login component gained significant traction and ranked among the Top 5 most used components in the Kony Marketplace, demonstrating its quality and widespread adoption across the customer base.